

Modelagem de Dados NoSQL

Prof. Msc. Edgard Devanir Amoroso

Hackolade

- Hackolade é uma ferramenta para modelagem ágil de dados visuais para bancos de dados JSON e NoSQL.
- Ele fornece visualização gráfica de estruturas de dados complexas usando diagramas Entidade-Relacionamento para representar dados desnormalizados de maneira amigável.
- Antes de partir para utilizar o MongoDB, iremos saber como modelar os dados a serem manipulados nesse banco de dados

Hackolade

Hackolade

File Edit View Actions Repository Tools Help

{ }

🔗

📁

⚙️



Common tasks

**New model**
Create a new Hackolade model

**Open model**
Open an existing Hackolade model

**Reverse-Engineer target**
This feature is only available in the Professional Edition

**Compare and merge models**
This feature is only available in Hackolade Pro edition

Recent models

— □ ×



Resources

**User manual**
Online documentation

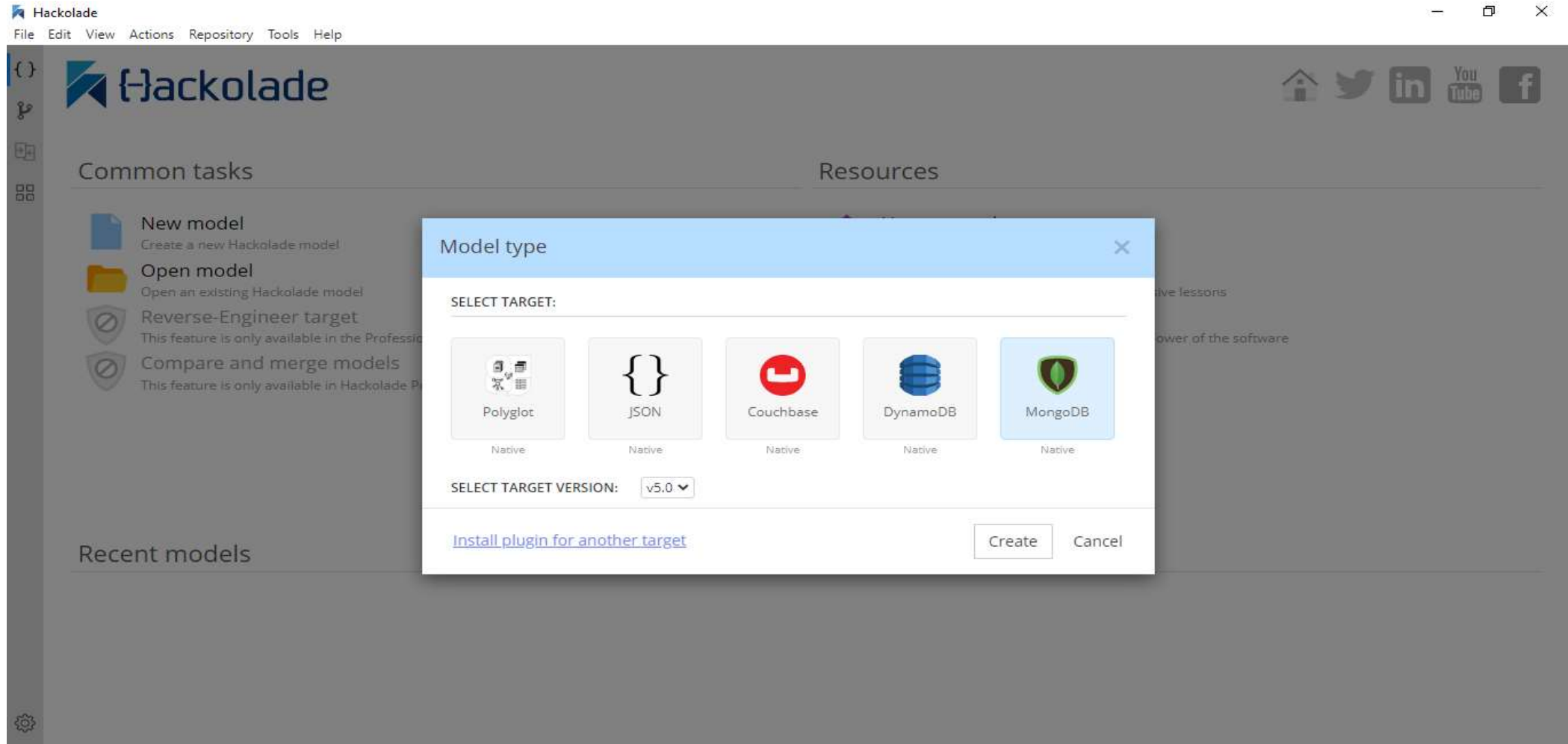
**Tutorial**
Get started at your own pace with progressive lessons

**Sample models**
Learn with examples how to leverage the power of the software

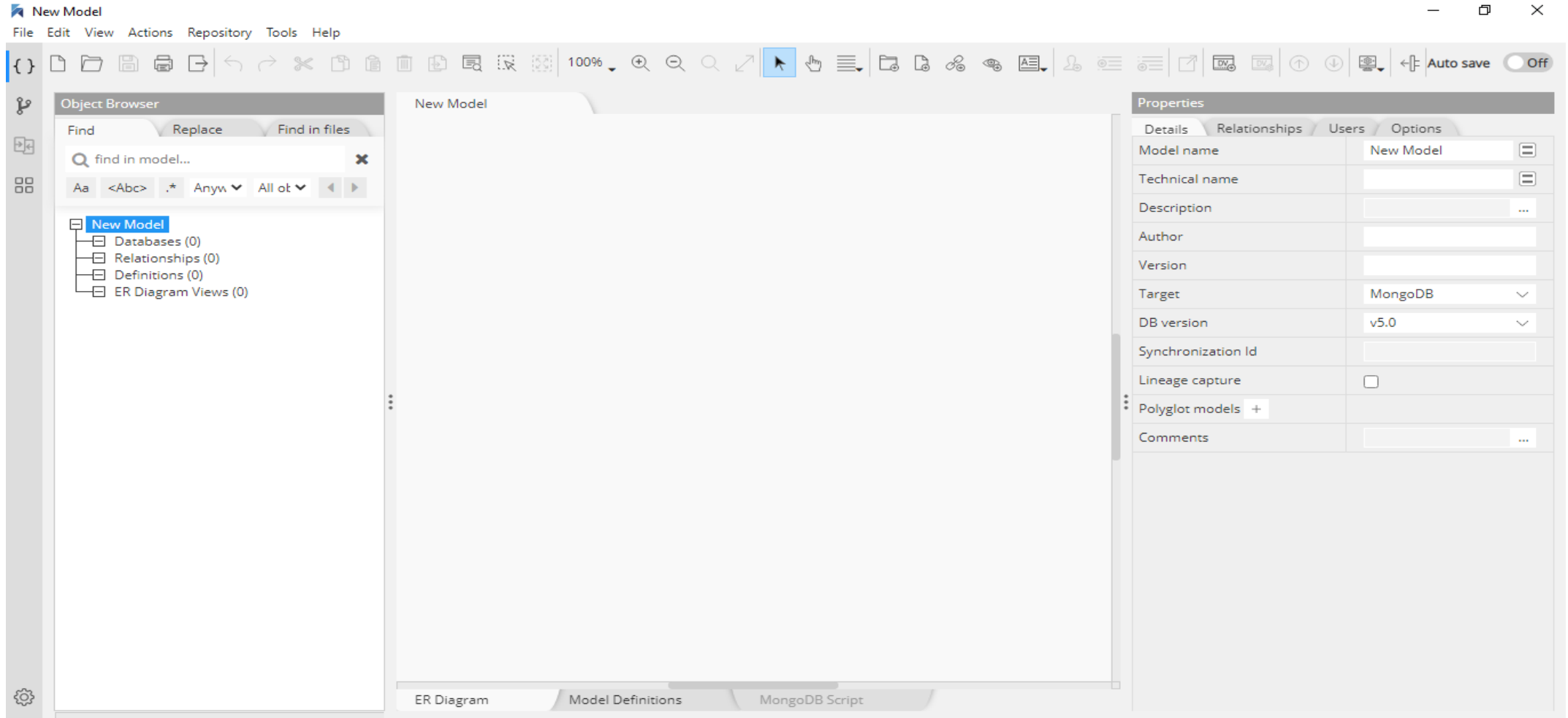
**Support**
Submit ticket or ask a question

Recent places

Hackolade



Hackolade



Hackolade

- Link para download
<https://hackolade.com/>

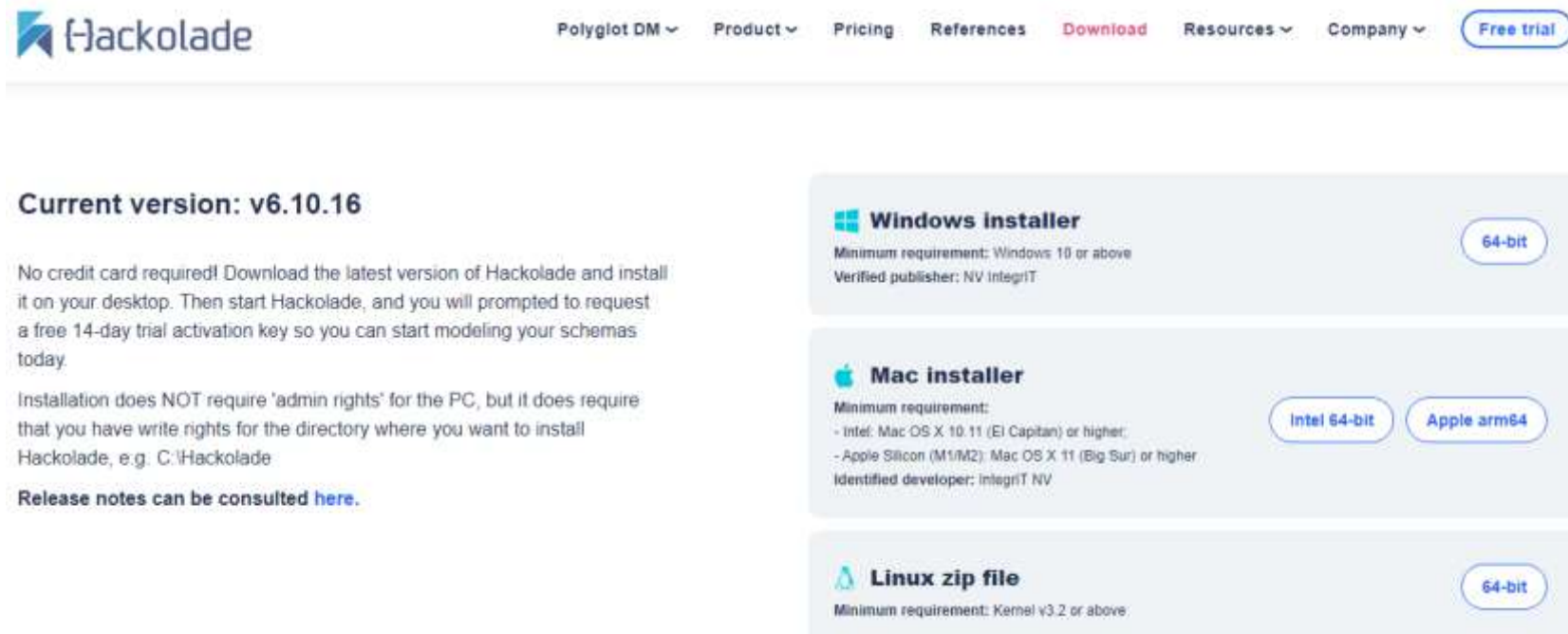


The screenshot shows the Hackolade website homepage. At the top is the Hackolade logo and a navigation menu with links: Polyglot DM, Product, Pricing, References, Download, Resources, and Company. A 'Free trial' button is on the right. The main heading is 'Polyglot Data Modeling' with a subtitle 'for NoSQL databases, storage formats, REST APIs, and JSON in RDBMS'. Below this is a large blue arrow pointing to a 'Start your FREE trial' button, with the text 'Free 14-day trial. No credit card. Easy setup.' underneath. Further down, a section titled 'Make business sense of your data with Metadata-as-Code and visual schema design for data storage and data exchanges' lists various database integrations in a grid:

NoSQL Document	NoSQL Columnar	NoSQL Property Graph	Semantic Web RDF Triple Stores	NoSQL Key-Value	NoSQL Multi-model	Git Repository Integrations
MongoDB	Cassandra DataStax	Neo4j	AllegroGraph	DynamoDB	ArangoDB	Azure DevOps Repos
Couchbase	HBase	JanusGraph	graphDB Ontotext	Redis	Cosmos DB	Bitbucket

Hackolade

- Link para download
<https://hackolade.com/>



The screenshot shows the Hackolade website's download page. At the top, there is a navigation bar with links for Polyglot DM, Product, Pricing, References, Download (highlighted in red), Resources, and Company, along with a 'Free trial' button. The main content area features the Hackolade logo and the text 'Current version: v6.10.16'. Below this, a paragraph states: 'No credit card required! Download the latest version of Hackolade and install it on your desktop. Then start Hackolade, and you will prompted to request a free 14-day trial activation key so you can start modeling your schemas today.' Another paragraph mentions: 'Installation does NOT require 'admin rights' for the PC, but it does require that you have write rights for the directory where you want to install Hackolade, e.g. C:\Hackolade'. A link for 'Release notes can be consulted here.' is provided. The download options are listed in three sections: 'Windows installer' (64-bit), 'Mac installer' (Intel 64-bit and Apple arm64), and 'Linux zip file' (64-bit). Each section includes minimum requirements and the verified publisher (NV IntegrIT).

Current version: v6.10.16

No credit card required! Download the latest version of Hackolade and install it on your desktop. Then start Hackolade, and you will prompted to request a free 14-day trial activation key so you can start modeling your schemas today.

Installation does NOT require 'admin rights' for the PC, but it does require that you have write rights for the directory where you want to install Hackolade, e.g. C:\Hackolade

Release notes can be consulted [here](#).

Windows installer
Minimum requirement: Windows 10 or above
Verified publisher: NV IntegrIT
64-bit

Mac installer
Minimum requirement:
- Intel: Mac OS X 10.11 (El Capitan) or higher;
- Apple Silicon (M1/M2): Mac OS X 11 (Big Sur) or higher
Identified developer: IntegrIT NV
Intel 64-bit Apple arm64

Linux zip file
Minimum requirement: Kernel v3.2 or above
64-bit

Hackolade

- Quando a instalação é finalizada selecionar a opção Community para obter a licença
- Após as informações solicitadas, é remetido para a empresa um pedido de licença sem custos
- Você receberá um e-mail, após análise da empresa, a licença para que você possa utilizar livremente.

Tipos de Bancos de Dados NoSQL

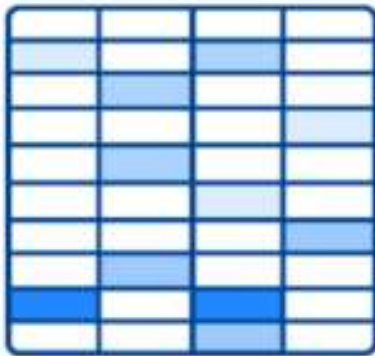
Banco de Dados NoSQL

- Temos vários tipos de Banco de Dados NoSQL, conforme segue:
 - Em colunas
 - Grafos
 - Chave-valor
 - Documentos
- A seguir apresentamos alguns desse banco de dados

Colunas

- Banco de dados colunares possuem o armazenamento orientado a colunas, o que influencia significativamente na sua performance, já que diminuem a quantidade de dados que você precisará carregar no disco.

Wide-column



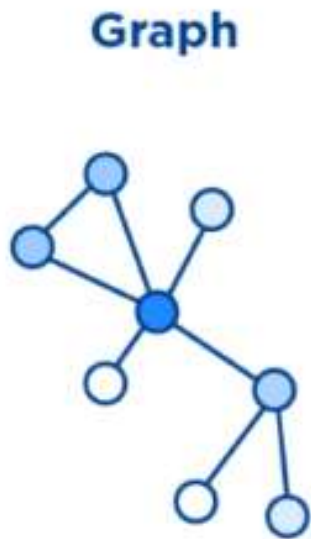
A diagram illustrating a wide-column database structure. It shows a grid of 10 rows and 4 columns. The grid is divided into four quadrants by a vertical line between the second and third columns. The top-left quadrant (2 columns x 5 rows) and the bottom-right quadrant (2 columns x 5 rows) are shaded light blue. The top-right quadrant (2 columns x 5 rows) and the bottom-left quadrant (2 columns x 5 rows) are white. This represents a columnar storage layout where data is organized by columns.



cassandra

Orientado a Grafos

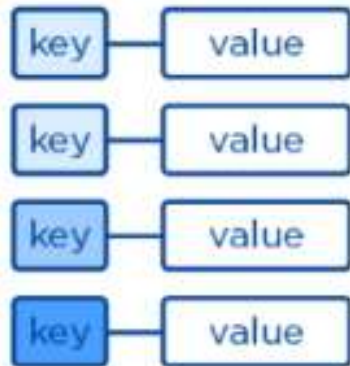
- Com a armazenagem de documentos em forma de grafos, no banco de dados orientado a grafos os dados são predispostos no formato de arcos conectados por arestas.



Chave-Valor

- Bastante simples, o banco de dados do tipo chave-valor é formado apenas por pares de chaves com valores associados, permitindo obter os valores quando uma consulta é realizada a uma chave.

Key-Value

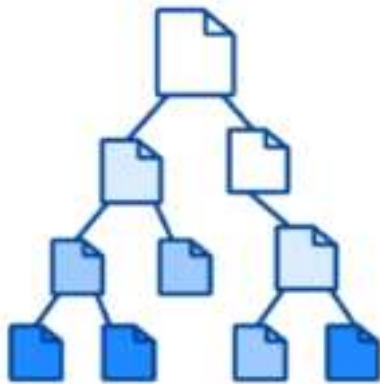


redis

Orientado a Documentos

- Banco de dados multiplataforma orientado ao documento que fornece alta performance, alta disponibilidade e fácil escalabilidade. MongoDB funciona no conceito de coleções de documentos. Esse será o banco de dados utilizado nessa disciplina

Document



MongoDB

MongoDB

MongoDB Compass - localhost:27017/MongoDBA.Funcionarios

Connect Edit View Collection Help

localhost:27017

Documents
MongoDBA.Funcionarios

My Queries

Databases

Search

Animais

Zoologico

MongoDBA

Funcionarios

Oscar

Premiações

admin

config

local

MongoDBA.Funcionarios

2 DOCUMENTS 1 INDEXES

Documents Aggregations Schema Indexes Validation

Filter `{"_id.idade" : "45" }` Explain Reset Find `</>` Options

Project `{"_id.nome": 1, "_id.setor": 1}`

Sort `{"_id.nome": 1}` MaxTimeMS 60000

Collation `{ locale: 'simple' }` Skip 0 Limit 0

EXPORT DATA

1 - 1 of 1

`{ "_id": { "nome": "José Silva", "setor": "RH" } }`

Mongo DB

Create Database

- Para criar um banco de dados basta utilizar o comando “use database_name”, o comando irá criar um novo banco de dados se caso ele não existir, de outra forma ele irá retornar o banco de dados existente.

A sintaxe básica para a utilização do USE_DATABASE é a seguinte:

use DATABASE_NAME

use Zoologico

Mongo DB

Show

Para verificar quais os bancos existentes, a instrução é a seguinte:

```
show dbs
```

Mongo DB

Drop Database

- Para excluir um banco de dados basta utilizar o comando `"drop database_name"`

A sintaxe básica para a utilização do USE_DATABASE é a seguinte:

db.dropDatabase()

```
db.dropDatabase()>{"dropped":"Zoologico","ok":1}>
```

Mongo DB

Coleções (Collection)

- É um grupo de documentos MongoDB. É o equivalente de uma tabela de RDBMS. Uma coleção existe dentro de um único banco de dados.
- Coleções não impõem um esquema. Documentos dentro de uma coleção pode ter diferentes campos. Normalmente, todos os documentos em uma coleção são propositalmente semelhantes ou afins.

Mongo DB

Create Collection

- Para esta tarefa, utilizaremos o comando `db.createCollection(name, options)`, onde "name" é o nome da Collection e "options" é utilizada para especificar as configurações da Collection.

Mongo DB

Create Collection

- A sintaxe básica do método `createCollection ()` é como se segue:

```
>use Zoologico
```

```
switched to db Zoologico
```

```
>db.createCollection("animais"){ "ok":1}>
```

Mongo DB

Create Collection

- Você pode verificar a criação da collection com o comando a seguir:

```
>show collections  
mycollection  
system.indexes
```

Collection

Field	Type	Description
capped	Boolean	(Opcional) Se "true", habilita uma "capped collection". Capped collection é uma coleção com tamanho fixo que sobrescreve automaticamente as atualizações antigas sempre que atinge o tamanho máximo. Se você especificar "true", será necessário especificar também o parametro "size".
autoIndexID	Boolean	(Opcional) Se true, o índice é criado automaticamente no campo _id. O Valor Default é "false".
size	number	(Opcional) Especifica o tamanho máximo em bytes para uma capped collection. Se capped is true, então será necessário especificar este campo também.
max	number	(Opcional) Especifica o numero máximo de documentos permitidos em uma the capped collection.

```
>db.createCollection("Zoologico",{ capped :true, autoIndexID :true, size :6142800, max :10000}){"ok":1}>
```


Mongo DB

Drop Collection

- Para excluir uma collection, usar o seguinte comando:

```
db.COLLECTION_NAME.drop()
```

```
db.Zoologico.drop()
```

Mongo DB

Documento

- Um documento é um conjunto de pares de valores-chave.
- Documentos têm esquema dinâmico.
- Esquema dinâmico significa que os documentos na mesma coleção não precisam ter o mesmo conjunto de campos ou estrutura e campos comuns em documentos de uma coleção podem conter diferentes tipos de dados.

Documento

```
{
  _id:ObjectId(7df78ad8902c)
  title:'MongoDB - Guia Rapido',
  description:'MongoDB - Guia Rapido',
  by:'MongoDBWise',
  url:'http://www.mongodbwise.wordpress.com',
  tags:['mongodb','database','NoSQL'],
  likes:100,
  comments:[{
    user:'user1',
    message:'My first comment',
    dateCreated:newDate(2014,5,21,2,15),
    like:0},{
    user:'user2',
    message:'My second comments',
    dateCreated:newDate(2014,5,21,7,45),
    like:5}]]}
```

Mongo DB

Insert

- Para inserir um documento, usar o seguinte comando:

```
>db.COLLECTION_NAME.insert(document)
```

Mongo DB

Insert

- Exemplo:
use zoologico
db.animais.insert({nome: "Tucano", especie: "ave"})
db.animais.insert({nome: "Periquito", especie: "ave"})
db.animais.insert({nome: "Papagaio", especie: "ave"})
db.animais.insert({nome: "Piranha", especie: "peixe"})
db.animais.insert({nome: "Dourado", especie: "peixe"})
db.animais.insert({nome: "Anta", especie: "mamífero"})

Mongo DB

Find

- O comando `find ()` irá exibir todos os documentos de uma forma não estruturada. Para exibir os resultados de uma forma estruturada, você pode usar o comando `pretty()`. Usar o seguinte comando:

```
>db.COLLECTION_NAME.find()
```

```
>db.mycol.find().pretty()
```

Mongo DB

Find

- Exemplo:

```
db.animais.find()
```

Mongo DB

Find (com clausula where)

Operação	Syntaxe	Exemplo	RDBMS Equivalente
Equality	{<key>:<value>}	db.mycol.find({"by":"tutorials point"}).pretty()	where by = 'tutorials point'
Less Than	{<key>:{ \$lt:<value>}}	db.mycol.find({"likes":{ \$lt:50}}).pretty()	where likes < 50
Less Than Equals	{<key>:{ \$lte:<value>}}	db.mycol.find({"likes":{ \$lte:50}}).pretty()	where likes <= 50
Greater Than	{<key>:{ \$gt:<value>}}	db.mycol.find({"likes":{ \$gt:50}}).pretty()	where likes > 50
Greater Than Equals	{<key>:{ \$gte:<value>}}	db.mycol.find({"likes":{ \$gte:50}}).pretty()	where likes >= 50
Not Equals	{<key>:{ \$ne:<value>}}	db.mycol.find({"likes":{ \$ne:50}}).pretty()	where likes != 50

Mongo DB

Find (com clausula and)

No comando find () se você passar várias chaves, separando-os por ',' então MongoDB irá tratar como condição AND. A sintaxe básica é mostrada abaixo:

```
>db.mycol.find({key1:value1, key2:value2}).pretty()
```

```
>db.Zoologico.find({key1:"peixe", key2:"ave"}).pretty()
```

Mongo DB

Update

No comando update () você pode mudar os dados de um documento. A sintaxe básica é mostrada abaixo:

```
db.COLLECTION_NAME.update(SELECTIOIN_CRITERIA, UPDATED_DATA)
```

```
db.animais.update( { especie: "ave" }, { $set:{especie: "ave tropical" }} )
```

Mongo DB

Save

O método `save()` substitui o documento existente com o novo documento passado pelo comando `save()`. A sintaxe básica é mostrada abaixo:

```
db.animais.save( { especie: "ave" }, { especie: "ave tropical" } )
```

```
db.mycol.save(  
  {  
    "_id" : ObjectId(5983548781331adf45ec7), ({nome: "Dourado Agua Doce",  
    especie: "peixe couro"}) } )
```

Mongo DB

Remove

O método `remove()` de MongoDB é usado para remover documentos da coleção. O método `remove()` aceita dois parâmetros. A sintaxe básica é mostrada abaixo:

```
db.COLLECTION_NAME.remove(DELETION_CRITTERIA)
```

Mongo DB

Limit Method

Para limitar os registros no MongoDB, você precisa usar o método `limit()`. O método `limit()` aceita um argumento de tipo de número, que é o número de documentos a serem exibidos. A sintaxe básica é mostrada abaixo:

```
db.COLLECTION_NAME.find().limit(NUMBER)
```

Mongo DB

Sort

O método `sort()` aceita uma lista de documentos que contém campos, juntamente com a sua ordem de classificação. Para especificar a ordem utiliza-se 1 e -1 como classificação. 1 é usado para a ordem crescente enquanto -1 é usado para a ordem decrescente.

A sintaxe básica é mostrada abaixo:

```
db.COLLECTION_NAME.find().sort({KEY:1})
```

Mongo DB

Ensure index()

Índices servem para apoiar a resolução eficiente de consultas. Sem índices, o MongoDB deverá digitalizar todos os documentos de uma coleção para depois selecionar os documentos que correspondem a instrução de consulta. Essa verificação é altamente ineficiente e exigem muito do mongod para processar um grande volume de dados.

Índices são estruturas de dados especiais, que armazenam uma pequena parte do conjunto de dados em um formato mais prático. O índice armazena o valor de um campo específico ou um conjunto de campos, ordenados pelo valor do campo, tal como especificado no índice.

Mongo DB

Ensure index()

Aqui chave é o nome do registro em qual você deseja criar o índice e 1 é para ordem crescente. Para criar o índice em ordem decrescente você precisa usar -1.

A sintaxe básica é mostrada abaixo:

```
db.COLLECTION_NAME.ensureIndex({KEY:1})
```


Ensure index() Lista opcionais

Parameter	Type	Description
background	Boolean	Constrói o índice em segundo plano para que a construção de um índice não bloqueie outras atividades de banco de dados. Especifique true para construir em segundo plano. O valor padrão é false .
unique	Boolean	Cria um índice exclusivo para que a coleção aceite a inserção de documentos em que a chave de índice ou as chaves que correspondam a um valor existente no índice. Especifique true para criar um índice exclusivo. O valor padrão é false .
name	string	O nome do índice. Se não for especificado, o MongoDB gera um nome de índice concatenando os nomes dos campos indexados e a ordem de classificação.
dropDups	Boolean	Cria um índice exclusivo em um campo que pode ter duplicatas. Índices MongoDB registram apenas a primeira ocorrência de uma chave e removem de todos os documentos da coleção que contêm ocorrências subsequentes desta chave. Especifique true para criar um índice único. O valor padrão é false .
sparse	Boolean	Se for “ true ”, o índice só faz referência a documentos com o campo especificado. Estes índices usam menos espaço, mas se comportam de forma diferente em algumas situações (particularmente os tipos). O valor padrão é false .
expireAfterSeconds	integer	Especifica um valor, em segundos, como um TTL para controlar quanto tempo MongoDB retém documentos nesta coleção.
v	index version	O número da versão do índice. A versão índice padrão depende da versão do mongod em execução durante a criação do índice.
weights	document	O peso é um número que varia de 1 a 99.999 e denota a importância do campo em relação aos outros campos indexados em termos de pontuação.
default_language	string	Para um índice de texto, a linguagem que determina a lista de palavras de parada e as regras para a stemmer e tokenizador . O isenglish é valor padrão.
language_override	string	Para um índice de texto, especifique o nome do campo no documento que contém, a linguagem para substituir o idioma padrão. O valor padrão é a linguagem.

Mongo DB

Aggregate()

Operações de agregações (Aggregation) processa todos os registros de dados e retorna o resultado calculado. Grupo de operações de agregação de valores a partir de vários documentos juntos podem executar uma variedade de operações nos dados agrupados para retornar um único resultado. Os comandos SQL "count (*)" com "grupo by" seria equivalentes aos grupos de agregação no MongoDB. Para a agregação em mongodb você deve usar o método aggregate().

A sintaxe básica é mostrada abaixo:

```
db.COLLECTION_NAME.aggregate(AGGREGATE_OPERATION)
```

Aggregate() Lista opcionais

Expression	Description	Example
\$sum	Soma-se o valor definido a partir de todos os documentos da coleção.	db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$sum : "\$likes"}}}])
\$avg	Calcula a média de todos os valores dados a partir de todos os documentos da coleção.	db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$avg : "\$likes"}}}])
\$min	Obtém o mínimo dos valores correspondentes de todos os documentos da coleção.	db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$min : "\$likes"}}}])
\$max	Obtém o máximo dos valores correspondentes de todos os documentos da coleção.	db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$max : "\$likes"}}}])
\$push	Insere o valor de uma matriz no documento resultante.	db.mycol.aggregate([{\$group : {_id : "\$by_user", url : {\$push : "\$url"}}}])
\$addToSet	Insere o valor de uma matriz no documento resultante, mas não cria duplicatas.	db.mycol.aggregate([{\$group : {_id : "\$by_user", url : {\$addToSet : "\$url"}}}])
\$first	Obtém o primeiro documento dos documentos de origem de acordo com o agrupamento. Normalmente, isso só faz sentido em conjunto com alguns anteriormente aplicada "\$sort"-stage.	db.mycol.aggregate([{\$group : {_id : "\$by_user", first_url : {\$first : "\$url"}}}])
\$last	Obtém o último documento dos documentos de origem de acordo com o agrupamento. Normalmente, isso só faz sentido em conjunto com alguns anteriormente aplicada "\$sort"-stage.	db.mycol.aggregate([{\$group : {_id : "\$by_user", last_url : {\$last : "\$url"}}}])

Mongo DB

Replication()

Replicação é o processo de sincronização de dados entre vários servidores. A replicação oferece redundância e aumenta a disponibilidade de dados com várias cópias dos dados em diferentes servidores de banco de dados. A replicação de um banco de dados além de proteger da perda de um único servidor, também permite que você recupere devido a uma falha de hardware ou interrupções de serviço. Com as cópias adicionais dos dados, você pode dedicar uma para recuperação de desastres, relatórios, ou backup.

Mongo DB

Replication()

Por que replicação?

- Para manter seus dados seguros
- Alta disponibilidade de dados (24×7)
- Recuperação de Desastres
- Falta de tempo de inatividade para manutenção (como backups, reconstrução de índices, compactação)
- Escala de Leitura (cópias extras para ler)
- Conjunto de réplicas transparentes para o aplicativo

Mongo DB

Replication()

Como a replicação trabalha no MongoDB

MongoDB obtêm sua replicação através da utilização do conjunto de réplicas. Um conjunto de réplicas é um grupo de instances mongod que hospedam o mesmo conjunto de dados. Em uma réplica, um dos nós é o nó principal, que recebe todas as operações de escrita. Todas as outras instancias, secundárias, aplicam operações do primário de modo que eles tenham o mesmo conjunto de dados. Um conjunto de réplicas (Replica Set) pode ter apenas um nó principal.

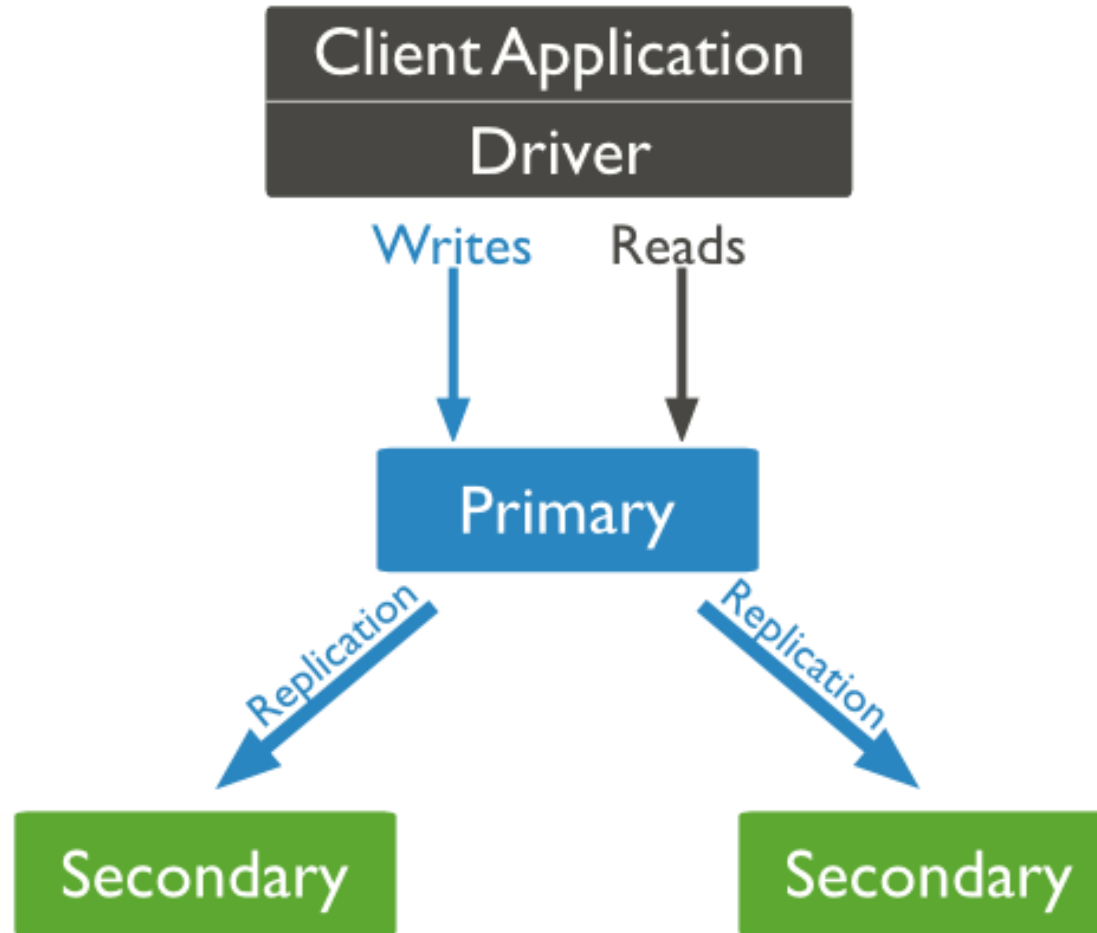
Mongo DB

Replication()

Como a replicação trabalha no MongoDB

- Conjunto de réplicas é um grupo de dois ou mais nós (geralmente são necessárias 3 nós no mínimo).
- Em um conjunto de réplicas, um nó é o nó principal e nós restantes são secundários.
- Todos os dados replicam do primário para o nó secundário.
- No momento do failover automático ou manutenção, a eleição estabelece qual será o primário e um novo nó principal é eleito.
- Após a recuperação do nó que falhou, ele novamente irá se juntar ao conjunto de réplicas e trabalhará como um nó secundário.

Mongo DB



Mongo DB

Backup

Para criar um backup de banco de dados no MongoDB você deve usar o comando `mongodump`. Este comando irá copiar todos os dados de seu servidor no diretório de dump. Há muitas opções disponíveis através do qual você pode limitar a quantidade de dados ou criar backup do seu servidor remoto.

A sintaxe do comando é a seguinte:

```
mongodump
```

Backup

Syntax	Description	Example
<code>mongodump --host HOST_NAME --port PORT_NUMBER</code>	Este comando irá fazer o backup todos os bancos de dados especificados na Instance mongod.	<code>mongodump --host tutorialspoint.com --port 27017</code>
<code>mongodump --dbpath DB_PATH --out BACKUP_DIRECTORY</code>		<code>mongodump --dbpath /data/db/ --out /data/backup/</code>
<code>mongodump --collection COLLECTION --db DB_NAME</code>	Este comando irá fazer o backup somente da collection do banco de dados, especificada.	<code>mongodump --collection mycol --db test</code>

Mongo DB

Restore

O comando para restaurar dados de backup do MongoDB é o mongorestore. Este comando irá restaurar todos os dados do diretório de backup.

A sintaxe do comando é a seguinte:

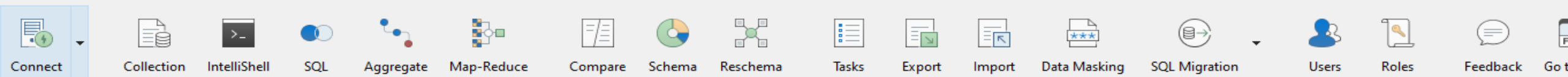
```
mongorestore
```

Robot3T

Robot3T

Studio 3T for MongoDB - Full product trial

File Edit Database Collection Index Document GridFS View Help



Open connections

Search open connections (Ctrl+F) aA

▼ Local - imported on 13 de ago. de 2023 localhost:27017

- > Animais
- > MongoDBA
- > Oscar
- > admin
- > config
- > local

Operations

Quickstart

Welcome to Studio 3T - Full product trial

Get started with the tool using our detailed [Knowledge Base](#).

Recent Connections



Local - imported on 13 de ago. de 2023
(localhost:27017)
Last accessed moments ago

Open Connection Manager

+ Create a new connection

Create a free MongoDB cluster in the cloud

Quick Options

Theme (requires restart): Same as system ▼

- ☒ Show What's New tab after updating Studio 3T
- ☒ Automatically open Connection Manager at startup

Tasks

Tasks is a feature that helps you automate common tasks.

0 tasks set.

Open Task Manager

Create a new task

Help and Learning

- Join our free live webinars
- Getting started
- Knowledge base
- Free MongoDB courses
- Join the 3T community
- Studio 3T features

Dúvidas?

